

Focus on Functional Chaos Expansion in OpenTURNS

Methodologie and OpenTURNS features

OpenTURNS Users' Day - 12 June, 2026

From the OpenTURNS 1.27 documentation

Plan

- 1 Motivation
- 2 Methodology : step by step
- 3 New strategy : the Ishigami example

Context and motivation

Problem

Let $\mathbf{X} \in \mathbb{R}^d$ be a random vector with density $\mu_{\mathbf{X}}$. The output random vector is $Y = g(\mathbf{X})$, where g might be costly. **We want to seek the distribution of Y .**

Monte Carlo simulations

- Easy to implement
- Slow convergence : $\mathcal{O}(N^{-1/2})$
- Requires a high calls number N of g

Alternative : meta model

- Replace g by an analytical function
- Easy and fast to evaluate

Functional Chaos Expansion

We project the model g in a suitable functional space, such as the Hilbert space $L^2(\mu_{\mathbf{X}})$ of square-integrable functions with respect to $\mu_{\mathbf{X}}$.

Methodology : step by step

Choice 1 : an approximation space $\mathcal{P}$

The approximation space $\mathcal{P}$ must be such that $\overline{\mathcal{P}} = L^2(\mu_X)$.

Choice 2 : a sequence of nested approximation spaces $(\mathcal{P}_n)_{n \in \mathbb{N}}$

The sequence of nested sub-spaces $\mathcal{P}_n$ such that $\mathcal{P} = \bigcup_{n \in \mathbb{N}} \mathcal{P}_n$. This implies that $\forall g \in L^2(\mu_X)$, there exists $(h_n)_{n \in \mathbb{N}}$, $h_n \in \mathcal{P}_n$ such that $\lim_{n \rightarrow \infty} \|g - h_n\|_{L^2(\mu_X)} = 0$.

Choice 3 : a basis $(\psi_k)_{k \in I_n}$ of $\mathcal{P}_n$

Then we have $\mathcal{P}_n = \text{span}(\psi_k)_{k \in I_n}$ where I_n is a finite index set. The increasing bases of $\mathcal{P}_n$ are ordered by an *enumerate function* that create bijections between $\mathbb{N}$ and $\mathbb{N}^{\dim(X)}$

Choice 4 : a specific space $\mathcal{P}_n$

The model g is approximated in the chosen subspace $\mathcal{P}_n$ by $\tilde{g}$:

$$\tilde{g} = \sum_{k \in I_n} a_k \psi_k = \arg \min_{h_n \in \mathcal{P}_n} \|g - h_n\|_{L^2(\mu_X)}^2 \quad (1)$$

Methodology : step by step

Choices 1 and 2 : $\mathcal{P}$ and $(\mathcal{P}_n)_{n \in \mathbb{N}}$

The approximation space $\mathcal{P}$ must be such that $\overline{\mathcal{P}} = L^2(\mu_{\mathbf{X}})$ and $\mathcal{P} = \bigcup_{n \in \mathbb{N}} \mathcal{P}_n$.

$\mathcal{P}$ is the set of multivariate polynomials ...
provided that $\overline{\mathcal{P}} = L^2(\mu_{\mathbf{X}})$!

- the measure $\mu_{\mathbf{X}}$ must have finite moments of all orders :
 $\mathbb{E}[X^n] = \langle X^n, 1 \rangle_{L^2(\mu_{\mathbf{X}})} < \infty$: ok for a bounded $\mu_{\mathbf{X}}$... but not for the Cauchy!
- it is not sufficient : the logNormal!

$\mathcal{P}$ is a non-polynomial approximation space : e.g : space spanned by trigonometric polynomials (Fourier space) or the Haar wavelets

and if $\overline{\mathcal{P}} \not\subseteq L^2(\mu_{\mathbf{X}})$?

Use an **isoprobabilistic transformation T** :
 $\mathbf{U} = T(\mathbf{X}) \sim \mu_{\mathbf{U}}$ such that $\overline{\mathcal{P}} = L^2(\mu_{\mathbf{U}})$.
Then consider $h = g \circ T^{-1} \in L^2(\mu_{\mathbf{U}})$.
Projecting h onto $(\varphi_k)_{k \in I_n}$ of $\mathcal{P}_n$ orthogonally
w.r.t. $\mu_{\mathbf{U}} \iff$ projecting g on $(\varphi_k \circ T)_{k \in I_n}$
orthogonally w.r.t. $\mu_{\mathbf{X}}$.

Methodology : step by step

Choice 3 : a basis $(\psi_k)_{k \in I_n}$ of $\mathcal{P}_n$

The $(\mathcal{P}_n)_{n \in \mathbb{N}}$ are nested : the basis of $\mathcal{P}_{n+1}$ is constructed from that of $\mathcal{P}_n$ by enrichment. Several choices of basis are possible.

A basis $\perp \mu_X$

- If μ_X has independent components, we can use the tensor product of univariate basis $\perp$ to each component of μ_X .
- If μ_X has dependent components, we can use an isoprobabilistic transformation T that maps μ_X into μ_U with independent marginals.

A basis NOT $\perp \mu_X$ but $\perp$ instrumental measure $\tilde{p} \neq \mu_X$ such that $\text{supp}(\tilde{p}) \supset \text{supp}(\mu_X)$, and $\tilde{p}$ has independent components.
This is a **new OpenTURNS strategy**.

Domination strategy

The *Domination strategy* projects g orthogonally w.r.t. $\tilde{p}$ on a basis which is $\perp \tilde{p}$...

The choice of basis of $\mathcal{P}_n$ has a major impact on the conditioning of the least-squares problem. If the basis is $\perp \mu_X$, the design matrix is well-conditioned. If not, it might lead to numerical instabilities.

Methodology : step by step

Choice 4 : a specific space $\mathcal{P}_n$

The sub-space $\mathcal{P}_n$ is fixed or guided (model selection methods). According to the chosen basis, the coefficients are computed :

A basis $\perp \mu_X$, the coefficients $(a_k)_{k \in I_n}$ can be computed using the inner product :

$$a_k = \langle g, \psi_k \rangle_{L^2(\mu_X)} \quad (2)$$

This integral can be computed using a quadrature rule (class *IntegrationStrategy*). If we used an isoprobabilistic transformation T , then

$$a_k = \langle h, \psi_k \rangle_{L^2(\mu_U)} = \langle g, \psi_k \circ T \rangle_{L^2(\mu_X)} \quad (3)$$

A basis NOT $\perp \mu_X$, then the coefficients are computed solving the least-squares problem (optimal estimators).

OpenTURNS

- *IntegrationExpansion* : inner product,
- *LeastSquaresExpansion* : least squares estimators,
- *FunctionalChaosAlgorithm* : manage both methods.

OpenTURNS documentation

Theory and Examples :

An Open source initiative for the Treatment of Uncertainties, Risk/N Statistics

OpenTURNS 1.27 documentation - Contents - Theory - Meta modeling

Meta modeling

The current section is dedicated to the building of an analytical approximation of the model of a given model. Such an approximation is commonly referred to as a response surface in the literature. This methodology is relevant if each model evaluation is time consuming; indeed, once a surrogate model has been built up, the various propagation steps may be applied at a negligible cost.

Local approximation

- Taylor expansion

Least squares surrogate models

- Least squares meta models
- Sparse least squares meta model

Gaussian Process Regression

- Gaussian process regression

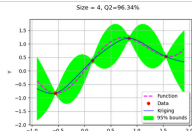
Functional chaos

- Functional Chaos Expansion
- Multivariate Orthonormal basis
- Univariate orthonormal polynomials
- Tensorized multivariate basis enumeration functions

Surrogate validation

- Validation and cross validation of metamodels

- Surrogate modeling
 - Linear regression
 - Polynomial chaos
 - Gaussian Process Regression
 - Fields surrogate models



API :

Functional chaos expansion

Main classes

<code>FunctionalChaosAlgorithm(*args)</code>	Functional chaos algorithm.
<code>LeastSquaresExpansion(*args)</code>	L2 approximation on an orthonormal basis using least-squares and a fixed basis.
<code>IntegrationExpansion(*args)</code>	L2 approximation on an orthonormal basis using least-squares and a fixed basis.

Construction of the truncated multivariate orthogonal basis

<code>AdaptiveStrategy(*args)</code>	Base class for the construction of the truncated multivariate orthogonal basis.
<code>CleaningStrategy(*args)</code>	Cleaning truncation strategy.
<code>FixedStrategy(*args)</code>	Fixed truncation strategy.

Projection method

<code>ProjectionStrategy(*args)</code>	Base class for the evaluation strategies of the approximation coefficients.
<code>IntegrationStrategy(*args)</code>	Integration strategy for the approximation coefficients.
<code>LeastSquaresStrategy(*args)</code>	Least squares strategy for the approximation coefficients.

Least squares algorithms to compute the coefficients

<code>ApproximationAlgorithm(*args)</code>	Approximation algorithm.
<code>ApproximationAlgorithmImplementationFactory(*args)</code>	Approximation algorithm factory base class.
<code>PenalizedLeastSquaresAlgorithmFactory(*args)</code>	Penalized least squares algorithm factory.
<code>PenalizedLeastSquaresAlgorithm(*args)</code>	Penalized least squares algorithm.
<code>LeastSquaresMetaModelSelectionFactory(*args)</code>	Least squares metamodel selection factory.
<code>LeastSquaresMetaModelSelection(*args)</code>	Least squares metamodel selection factory.

Model selection algorithm

<code>BasisSequenceFactory(*args)</code>	Basis sequence factory.
<code>LARS(*args)</code>	Least Angle Regression.

Model selection criteria

<code>FittingAlgorithm(*args)</code>	Fitting algorithm.
<code>CorrectedLeaveOneOut(*args)</code>	Corrected leave one out.
<code>KFold(*args)</code>	K-fold.

Least Squares problem resolution

Refer to **Least squares meta models**.

<code>LeastSquaresMethod(*args)</code>	Base class for least square solvers.
<code>CholeskyMethod(*args)</code>	Least squares solver using Cholesky decomposition.
<code>SVDMethod(*args)</code>	Least squares solver using SVD decomposition.
<code>QRMethod(*args)</code>	Least squares solver using the QR decomposition.
<code>SparseMethod(*args)</code>	Least squares solver using a sparse representation.
<code>DesignProxy(*args)</code>	Design matrix cache.

Results

<code>FunctionalChaosRandomVector(*args)</code>	Functional chaos random vector.
<code>FunctionalChaosResult(*args)</code>	Functional chaos result.
<code>FunctionalChaosSobolIndices(*args)</code>	Sensitivity analysis based on functional chaos expansion.
<code>FunctionalChaosValidation(*args)</code>	Validate a functional chaos metamodel.

Ishigami example

The Ishigami function

The Ishigami function of Ishigami & Homma (1990) is recurrent test case for sensitivity analysis methods and uncertainty. Let $a = 7$ and $b = 0.1$ (see Crestaux et al. (2007) and Marel et al. (2009)). We consider the function:

$$g(x_1, x_2, x_3) = \sin(x_1) + a \sin(x_2)^2 + b x_3^4 \sin(x_1)$$

for any $(x_1, x_2, x_3) \in [-\pi, \pi]^3$. We assume that the random variables X_1, X_2, X_3 are independent and have the uniform marginal distribution in the interval from $-\pi$ to π :

$$(X_1, X_2, X_3) \sim \mathcal{U}(-\pi, \pi).$$

We consider the random variable Y defined by:

$$Y = g(X_1, X_2, X_3)$$

Create a FCE for dependent inputs: transformation vs domination

In this example, we create a functional chaos expansion for the **Ishigami function** when the input distribution has dependent marginals.

Refer to **Functional Chaos Expansion** to learn more about functional chaos expansion.

We provide one input sample and one output sample of the Ishigami function. We build two meta models:

- Meta model 1: we use an isoprobabilistic transformation that maps the input distribution to another one with independent marginals.
- Meta model 2: we use the domination method: the basis of the projection space is not orthonormal to the input distribution.

Define the Ishigami model

```
from openturns.usecases import ishigami_function
import openturns as ot
import openturns.viewer as otv
```

We load the Ishigami model. The **IshigamiModel** data class contains the input distribution μ_X of the random vector $\mathbf{X} = (X_1, X_2, X_3)$ in `im.distribution`.

```
im = ishigami_function.IshigamiModel()
input_names = im.distribution.getDescription()
```

We want to introduce some dependence between the components. That is why we have to change the input distribution stored in the Ishigami model (which have independent components). We use a copula that links (X_1, X_2) with a **ClaytonCopula**. The last component X_3 is independent of (X_1, X_2) . The final 3d-copula is a **BlockIndependentCopula** copula. We keep the initial marginal distributions.

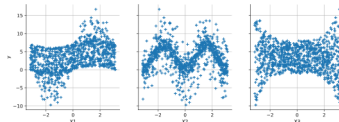
```
copula = ot.BlockIndependentCopula([ot.ClaytonCopula(1.5), ot.IndependentCopula(1)])
input_dist = ot.JointDistribution(
    [im.distribution.getMarginal(i) for i in range(im.dim)], copula
)
```

We generate an input sample from the input distribution and we compute the output sample from the Ishigami function that is contained in `im.model`.

```
sampleSize = 1000
inputTrain = im.distribution.getSample(sampleSize)
outputTrain = im.model(inputTrain)
```

We display the relationships between the outputs and the inputs

```
grid = ot.VisualTest.DrawPairsXY(inputTrain, outputTrain)
view = otv.View(grid, figure_kw={"figsize": (12.0, 4.0)})
```



Ishigami example : Transformation method vs New strategy

```
chaos_algo =  
ot.FunctionalChaosAlgorithm(inputTrain, outputTrain, input_dist)
```

As μ_X has dependent components, transformation method :

- isoprobabilistic transformation $\mu_X \rightarrow \mu_U = \otimes \mu_i$
- $\mathcal{P}$: set of polynomials + basis of $\mathcal{P}$:
 $(\otimes \psi_k^i)_k \perp \mu_U$
- $\mathcal{P}_n$: set of polynomials of total degree ≤ 10 + hyperbolic enumerate function : high marginal degrees first rather than mixed monomials
- least squares estimators : project $g \circ T^{-1}$ orthonormally w.r.t. to μ_U
- project g in $(\psi_k \circ T^{-1})_k$ orthonormally w.r.t. to μ_X

```
chaos_algo =  
ot.FunctionalChaosAlgorithm(inputTrain, outputTrain, input_dist)  
chaos_algo.setUseDomination(True)
```

- no isoprobabilistic transformation
- use the same $\mathcal{P}$ and $\mathcal{P}_n$
- use the same $(\psi_k)_k$ which is NOT $\perp \mu_X$
- least squares estimators
- project g in $(\psi_k)_k$ orthonormally w.r.t. to μ_X

Ishigami example

Transformation method

We create the functional chaos algorithm.

```
chaos_algo = ot.FunctionalChaosAlgorithm(inputTrain, outputTrain, input_dist)
chaos_algo.setUseDomination(False)
chaos_algo.run()
```

We get the result and the resulting meta model.

```
chaos_result = chaos_algo.getResult()
metamodel = chaos_result.getMetaModel()
```

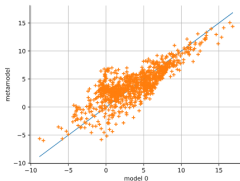
In order to validate the meta model, we generate a test sample.

```
n_valid = 1000
inputTest = input_dist.getSample(n_valid)
outputTest = in.model(inputTest)
metamodel_predictions = metamodel(inputTest)
```

We draw the validation graph and we get the R^2 score: the meta model is of poor quality.

```
val = ot.MetaModelValidation(outputTest, metamodel_predictions)
r2Score = val.computeR2Score()[0]
print(f"r2Score with Transformation method = {r2Score:.6f}")
graph = val.drawValidation()
graph.setTitle(f"R2={r2Score * 100:.2f}%, use domination = false")
view = otv.View(graph)
```

R2=64.71%, use domination = false



New strategy

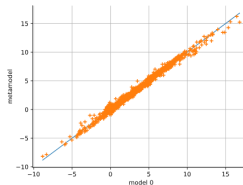
Now, we want to use the domination method, which means that the basis created by the adaptive strategy is used to project the model. This basis is not orthonormal to μ_k .

We use the `setUseDomination()` method. We implement the same steps as before, until the validation graph.

```
chaos_algo.setUseDomination(True)
chaos_algo.run()

chaos_result = chaos_algo.getResult()
metamodel_dom = chaos_result.getMetaModel()
metamodel_dom_predictions = metamodel_dom(inputTest)
val = ot.MetaModelValidation(outputTest, metamodel_dom_predictions)
r2Score = val.computeR2Score()[0]
print(f"r2Score with Domination method = {r2Score:.6f}")
graph = val.drawValidation()
graph.setTitle(f"R2={r2Score * 100:.2f}%, use domination = true")
view = otv.View(graph)
```

R2=98.95%, use domination = true



Conclusion

The **New strategy** uses the multivariate tensorized basis $(\psi_k)_k$ which is able to capture the tensorized structure of the Ishigami model, while the **Transformation method** uses $(\psi_k \circ T^{-1})_k$ which is not tensorized due to T .